

Ejercicios Flash para el Final de EDA

Guia formal de teoria aplicada, soluciones y complejidades

Como usar esta guia

Este documento reemplaza la version anterior de `ejercicios_flash.pdf`. Esta escrito para responder problemas teoricos tipo Raco: no basta nombrar una estructura; hay que justificar por que aplica, que guarda, como responde, por que es correcto y cuanto cuesta.

Orden de estudio si falta tiempo.

1. Lee primero la Seccion 1: plantilla de respuesta y mapa de decision.
2. Luego asegura las tres preguntas de la lista del profesor: Secciones 2, 3 y 4.
3. Despues strings: suffix array, LCP y aplicaciones (Secciones 6–9).
4. Finalmente dynamic e integer: rollback DSU, Euler-tour/link-cut, vEB/Y-fast/radix (Secciones 12–15).

Formato que debe sonar en el examen. “El problema es estatico/dinamico. La estructura correcta es X porque transforma la consulta en Y. Preproceso guardando Z. Para cada query hago estos pasos. La correctitud se basa en este invariante/propiedad. La complejidad temporal es ... y la memoria es ...”.

Contents

1	Plantilla formal y mapa de decision	4
1.1	Plantilla de respuesta para cualquier problema	4
1.2	Mapa de decision	4
2	Lista profesor 1: subsecuencia en un camino de arbol	5
2.1	Enunciado tipo	5
2.2	Reconocimiento	5
2.3	Estructuras que se guardan	5
2.4	Algoritmo	5
2.5	Correctitud	5
2.6	Complejidad	6
3	Lista profesor 2: ordenar nodos de un trie	6
3.1	Enunciado tipo	6
3.2	Idea	6
3.3	Algoritmo	6
3.4	Correctitud	6
3.5	Complejidad	6
4	Lista profesor 3: cantidad de caminos con peso $\leq C$	7
4.1	Enunciado tipo	7
4.2	Por que no basta LCA/HLD	7
4.3	Idea de centroid decomposition	7

4.4	Como contar pares con suma $\leq C$	7
4.5	Algoritmo completo	7
4.6	Correctitud	8
4.7	Complejidad	8
5	Metricas: bloque de emergencia de la lista	9
5.1	Definicion	9
5.2	Suma de metricas	9
5.3	Como verificar funciones rapidamente	9
6	Static RMQ y Sparse Table	10
6.1	Contexto	10
6.2	Definicion	10
6.3	Pseudocodigo	10
6.4	Correctitud	10
6.5	Complejidad	10
7	Suffix Array, Rank, LCP y Kasai	11
7.1	Definiciones basicas	11
7.2	Ejemplo: banana	11
7.3	Manber-Myers	11
7.4	Kasai	11
7.5	Formula clave con RMQ	11
7.6	Complejidad	11
8	Aplicaciones obligatorias de Suffix Array	12
8.1	Buscar y contar ocurrencias de un patron	12
8.2	Subcadenas distintas	12
8.3	Longest Common Substring	12
8.4	Comparar y ordenar subcadenas	12
8.5	Bordes de todas las subcadenas	12
9	Arboles estaticos: LCA, HLD, Imperial Roads y Fools and Roads	13
9.1	LCA con binary lifting	13
9.2	HLD	13
9.3	Imperial Roads	13
9.4	Fools and Roads	13
10	Dynamic Graphs: rollback DSU y estructuras dinamicas	14
10.1	Offline dynamic connectivity	14
10.2	Rollback DSU	14
10.3	Correctitud	14
10.4	Complejidad	14
10.5	Euler-tour tree vs link-cut tree	14
11	Integer structures	16
11.1	Modelo y problema predecesor	16

11.2 van Emde Boas	16
11.3 Y-fast trees	16
11.4 Fusion trees	16
11.5 Ordenamiento de enteros	17
12 Patrones de respuesta para problemas Racso	18
12.1 Si aparece un problema de strings	18
12.2 Si aparece un problema de arboles	18
12.3 Si aparece dynamic graph	18
12.4 Si aparece integer	18
13 Errores que quitan puntos	18
14 Resumen final de formulas y complejidades	19

1 Plantilla formal y mapa de decision

1.1 Plantilla de respuesta para cualquier problema

Para cada ejercicio, intenta llenar esta plantilla en papel:

1. **Objeto:** arreglo, string, arbol, bosque dinamico, grafo general, conjunto de enteros.
2. **Tipo:** estatico, incremental, decremental, fully dynamic, online u offline.
3. **Estructura:** sparse table, suffix array, LCA, HLD, centroid decomposition, rollback DSU, Euler-tour tree, link-cut tree, vEB/Y-fast, radix sort.
4. **Que guarda:** por ejemplo, LCP, ancestors 2^j , maximo en salto, componentes DSU, intervalos de vida de aristas, prefijos de un trie binario.
5. **Query/update:** traducir la pregunta al lenguaje de la estructura.
6. **Correctitud:** explicar el invariante que hace que la traduccion sea exacta.
7. **Complejidad:** preprocesamiento, query/update y memoria.

1.2 Mapa de decision

Frase del enunciado	Estructura	Complejidad tipica
Arreglo fijo, min/max/gcd en rangos	Sparse table	build $O(n \log n)$, query $O(1)$
Camino en arbol fijo	LCA / HLD	LCA $O(\log n)$, HLD $O(\log^2 n)$
Subarbol en arbol fijo	Euler tour timestamps	query/update como intervalo
Muchos patrones en texto fijo	Suffix array o Aho-Corasick	depende de patrones
Comparar/ordenar subcadenas	SA + LCP + RMQ	comparacion $O(1)$
Contar caminos en arbol con restriccion de distancia	Centroid decomposition	$O(n \log^2 n)$
Add/delete edges offline	Segment tree tiempo + rollback DSU	$O(m \log q)$ uniones
Bosque dinamico con path aggregate	Link-cut tree	$O(\log n)$ amortizado
Bosque dinamico con componente/subtree aggregate	Euler-tour tree	$O(\log n)$
Predecesor de enteros	vEB / Y-fast / Fusion	$O(\log w)$, $O(\log_w n)$
Ordenar enteros	Counting/radix/signature	puede romper $n \log n$

2 Lista profesor 1: subsecuencia en un camino de arbol

2.1 Enunciado tipo

Se tiene un arbol con N vertices. Cada vertice contiene una letra minuscula. En una consulta se dan U, V y una cadena S . Sea T la cadena obtenida al escribir, en orden, las letras del camino simple de U a V . Se debe decidir si S es subsecuencia de T .

2.2 Reconocimiento

El arbol es fijo y las queries son sobre caminos. La palabra clave es **camino en arbol estatico**. La estructura natural es HLD, porque descompone cualquier camino en $O(\log N)$ segmentos de cadenas pesadas. Como el alfabeto es pequeno, se puede guardar para cada letra las posiciones donde aparece en el arreglo lineal de HLD.

2.3 Estructuras que se guardan

- HLD: $parent[v]$, $depth[v]$, $head[v]$, $pos[v]$, $out[v]$.
- Arreglo base $B[pos[v]] = \text{letra del vertice } v$.
- Para cada letra $c \in \{a, \dots, z\}$, lista ordenada $L_c = \{pos[v] : B[pos[v]] = c\}$.

2.4 Algoritmo

Primero se descompone el camino $U \rightarrow V$ en segmentos sobre el arreglo base, respetando el orden real del camino. Hay dos partes:

$$U \rightarrow lca(U, V) \quad \text{y} \quad lca(U, V) \rightarrow V.$$

La primera parte se recorre hacia arriba; dentro de una cadena HLD eso puede significar posiciones decrecientes. La segunda parte se recorre hacia abajo; se guardan sus segmentos y se invierten al final.

Luego se consume la cadena S de izquierda a derecha. Si el caracter actual es $S[i]$, se busca en la lista $L_{S[i]}$ la primera aparicion valida dentro del segmento actual y despues de la posicion ya usada. En segmentos crecientes se usa **lower_bound**; en segmentos decrecientes se usa **upper_bound** y se retrocede.

```
query(U,V,S):
    segments = ordered_HLD_segments(U,V)  // each segment has [l,r] and direction
    i = 0
    for seg in segments:
        cur = seg.start
        while i < |S|:
            p = first_position_of_char_inside_segment_after(cur, S[i], seg)
            if p does not exist: break
            cur = next_position_after(p, seg.direction)
            i++
    return i == |S|
```

2.5 Correctitud

HLD no altera el camino; solo lo parte en segmentos contiguos de cadenas pesadas. Por tanto, concatenar los segmentos en el orden correcto produce exactamente T . Dentro de un segmento, las listas L_c permiten encontrar la primera aparicion de un caracter que todavia puede usarse.

El greedy de tomar la primera aparicion valida es correcto para subsecuencias: si existe una solucion que usa una aparicion posterior del caracter actual, reemplazarla por una aparicion anterior no invalida los caracteres restantes, porque deja un sufijo del camino igual o mas grande para continuar. Por induccion sobre los caracteres de S , si el algoritmo falla en algun caracter, ninguna subsecuencia completa existe.

2.6 Complejidad

Preprocesamiento HLD: $O(N)$. Construir listas por caracter: $O(N)$ tiempo y espacio. Una query usa $O(\log N)$ segmentos. Cada caracter consumido hace una busqueda binaria en una lista, costando $O(\log N)$. Por tanto:

$$T_{\text{query}} = O((|S| + \log N) \log N), \quad M = O(N).$$

Si se optimizan las listas por cadena pesada, puede bajarse constante, pero esta respuesta ya es claramente menor que recorrer todo el camino para cada consulta.

3 Lista profesor 2: ordenar nodos de un trie

3.1 Enunciado tipo

Dado un trie T , cada nodo u representa la cadena formada por las etiquetas de las aristas desde la raiz hasta u . Se pide ordenar los nodos por la comparacion lexicografica de sus cadenas en complejidad menor que $O(n^2)$.

3.2 Idea

En orden lexicografico, si una palabra A es prefijo propio de B , entonces $A < B$. Por eso un nodo debe salir antes que todos sus descendientes. Entre dos hijos de un mismo nodo, el caracter menor debe procesarse primero. Esto sugiere DFS preorder con hijos ordenados.

3.3 Algoritmo

```
lexicographic_trie_order(root):
    answer = []
    dfs(root)

dfs(u):
    append u to answer
    for character c in increasing order:
        if child[u][c] exists:
            dfs(child[u][c])
```

Si la raiz representa la cadena vacia y el problema pide ordenar todos los nodos, la raiz va primero. Si solo se piden cadenas no vacias, se omite la raiz del arreglo final.

3.4 Correctitud

Sea u un nodo y v un descendiente propio. La cadena de u es prefijo propio de la cadena de v ; por definicion lexicografica, $str(u) < str(v)$. Por eso visitar u antes que su subarbol es correcto.

Ahora considera dos nodos que pertenecen a subarboles de hijos distintos a y b de un mismo nodo, con etiquetas $c_a < c_b$. Sus cadenas comparten el prefijo del padre y difieren por primera vez en esas etiquetas, por lo que toda cadena del subarbol de a es menor que toda cadena del subarbol de b . Visitar hijos en orden creciente respeta ese criterio. Por induccion sobre la profundidad del trie, el DFS produce el orden lexicografico global.

3.5 Complejidad

Si el alfabeto es fijo, por ejemplo $\sigma = 26$, iterar todos los caracteres en cada nodo cuesta:

$$O(n\sigma) = O(n).$$

La memoria adicional es $O(h)$ por recursion, donde h es la altura del trie, o $O(n)$ en el peor caso. Si los hijos estan guardados en listas desordenadas, se ordenan por caracter y el costo total es $\sum_u \deg(u) \log \deg(u) \leq O(n \log n)$, todavia menor que $O(n^2)$.

4 Lista profesor 3: cantidad de caminos con peso $\leq C$

4.1 Enunciado tipo

Dado un arbol ponderado T , calcular la cantidad de caminos $u \rightsquigarrow v$ cuyo peso total sea menor o igual que una constante C . El algoritmo debe ser menor que $O(n^2)$.

4.2 Por que no basta LCA/HLD

LCA calcula el peso de un camino especifico en $O(\log n)$. Pero aqui se piden muchos caminos: en el peor caso hay $\Theta(n^2)$ pares (u, v) . Necesitamos contar pares sin enumerarlos. La estructura clasica para contar caminos con restricciones de distancia en arboles es **centroid decomposition**.

4.3 Idea de centroid decomposition

Un centroid c es un nodo cuya remocion deja componentes de tamano a lo mucho $n/2$. En cualquier nivel de la recursion:

- contamos todos los caminos que pasan por el centroid c ;
- restamos los pares que caen completamente dentro del mismo subarbol hijo, porque esos no pasan por c ;
- recursamos en cada componente restante.

4.4 Como contar pares con suma $\leq C$

Recolecta todas las distancias desde c a los nodos de la componente:

$$D = \{\text{dist}(c, x) : x \text{ en la componente}\}.$$

Ordena D . Con dos punteros:

```
count_pairs(D,C):
    sort(D)
    l = 0; r = |D|-1; ans = 0
    while l < r:
        if D[l] + D[r] <= C:
            ans += r-l
            l++
        else:
            r--
    return ans
```

Esto cuenta pares no ordenados $\{u, v\}$ con $\text{dist}(c, u) + \text{dist}(c, v) \leq C$.

4.5 Algoritmo completo

```
solve(component):
    c = centroid(component)
    mark c as removed
    all = [0] // distance from c to c
    for each neighbor subtree S of c not removed:
        temp = distances from c to nodes in S
        answer -= count_pairs(temp, C)
        append temp to all
    answer += count_pairs(all, C)
    for each remaining component S:
        solve(S)
```

4.6 Correctitud

Todo camino $u \rightsquigarrow v$ tiene un primer centroid en la recursion que separa a u y v en componentes distintas, o coincide con uno de los extremos. En ese nivel, el camino pasa por el centroid c , por lo que su peso es:

$$\text{dist}(u, v) = \text{dist}(c, u) + \text{dist}(c, v).$$

El conteo global `count_pairs(all,C)` cuenta ese par. Si ambos extremos estaban en el mismo subarbol de c , el camino no pasa por c , por eso se resta en `count_pairs(temp,C)` y se deja para la recursion dentro de ese subarbol. Asi, cada camino valido se cuenta exactamente una vez.

4.7 Complejidad

Cada nivel de centroid decomposition reduce el tamaño de la componente al menos a la mitad, por lo que hay $O(\log n)$ niveles por nodo. Si en cada nivel se ordenan listas, una cota simple es:

$$T(n) = O(n \log^2 n), \quad M(n) = O(n).$$

Esto cumple claramente menor que $O(n^2)$. Con tecnicas de merge o manteniendo listas ordenadas puede mejorarse a $O(n \log n)$, pero para respuesta de examen la version $O(n \log^2 n)$ es defendible.

5 Metricas: bloque de emergencia de la lista

5.1 Definicion

Una funcion $d : X \times X \rightarrow \mathbb{R}$ es metrica si cumple:

1. No negatividad: $d(x, y) \geq 0$.
2. Identidad: $d(x, y) = 0 \Leftrightarrow x = y$.
3. Simetria: $d(x, y) = d(y, x)$.
4. Desigualdad triangular: $d(x, z) \leq d(x, y) + d(y, z)$.

5.2 Suma de metricas

Si d_1, d_2 son metricas y $d_3(x, y) = d_1(x, y) + d_2(x, y)$, entonces d_3 es metrica. La no negatividad y simetria son inmediatas. Para identidad, si $d_3(x, y) = 0$, como ambos terminos son no negativos, $d_1(x, y) = d_2(x, y) = 0$, luego $x = y$. Para triangular:

$$d_3(x, z) = d_1(x, z) + d_2(x, z) \leq d_1(x, y) + d_1(y, z) + d_2(x, y) + d_2(y, z) = d_3(x, y) + d_3(y, z).$$

5.3 Como verificar funciones rapidamente

- $(x - y)^2$ no es metrica: $d(0, 2) = 4$, pero $d(0, 1) + d(1, 2) = 1 + 1 = 2$.
- $|x - y| + |f(x) - f(y)|$ suele ser metrica si incluye $|x - y|$: el segundo termino es una pseudometrica y el primero garantiza identidad.
- L_1, L_2 y L_∞ son metricas en $\mathbb{R}^d$.
- Pesos positivos de L_1 , por ejemplo $2|x_1 - y_1| + 3|x_2 - y_2|$, son metricas.
- $\sqrt{|x - y|}$ es metrica porque $\sqrt{a + b} \leq \sqrt{a} + \sqrt{b}$.
- $\min\{1, |x - y|\}$ es metrica: truncar una metrica por 1 preserva triangular.
- $\ln(1 + |x - y|)$ es metrica porque

$$\ln(1 + a + b) \leq \ln((1 + a)(1 + b)) = \ln(1 + a) + \ln(1 + b).$$

- $\frac{|x - y|}{1 + |x - y|}$ es metrica porque $f(t) = t/(1 + t)$ es creciente y subaditiva.

6 Static RMQ y Sparse Table

6.1 Contexto

RMQ significa Range Minimum Query. Dado un arreglo fijo A , una consulta pide el minimo en un rango. Si no hay updates, una sparse table es ideal para operaciones idempotentes:

$$\min(x, x) = x, \quad \max(x, x) = x, \quad \gcd(x, x) = x.$$

No sirve igual para suma con dos bloques solapados, porque $x + x \neq x$.

6.2 Definicion

$$st[k][i] = \min(A[i], A[i+1], \dots, A[i+2^k-1]).$$

La transicion parte el bloque en dos mitades:

$$st[k][i] = \min(st[k-1][i], st[k-1][i+2^{k-1}]).$$

Para una consulta cerrada $[l, r]$:

$$len = r - l + 1, \quad k = \lfloor \log_2 len \rfloor,$$

$$RMQ(l, r) = \min(st[k][l], st[k][r - 2^k + 1]).$$

Para una consulta medio abierta $[l, r)$, se usa $len = r - l$ y:

$$RMQ(l, r) = \min(st[k][l], st[k][r - 2^k]).$$

6.3 Pseudocodigo

```
build(A):
    lg[1] = 0
    for i = 2..n: lg[i] = lg[i/2] + 1
    for i = 0..n-1: st[0][i] = A[i]
    for k = 1..LG:
        for i = 0; i + 2^k <= n; i++:
            st[k][i] = min(st[k-1][i], st[k-1][i+2^(k-1)])

query(l,r): // closed interval
    len = r-l+1
    k = lg[len]
    return min(st[k][l], st[k][r-2^k+1])
```

6.4 Correctitud

Por induccion en k , $st[k][i]$ guarda el minimo del bloque de longitud 2^k . Para $k = 0$, el bloque tiene un solo elemento. Para $k > 0$, el bloque se divide en dos bloques correctos de longitud 2^{k-1} .

En una query, los dos bloques de longitud 2^k elegidos cubren todo el rango. Pueden solaparse, pero el minimo no cambia por contar un elemento dos veces. Por tanto, la respuesta es correcta.

6.5 Complejidad

$$T_{\text{build}} = O(n \log n), \quad T_{\text{query}} = O(1), \quad M = O(n \log n).$$

7 Suffix Array, Rank, LCP y Kasai

7.1 Definiciones basicas

Un **sufijo** de S es $S[i..n-1]$. Un **prefijo** es una parte inicial de una cadena. El suffix array SA guarda los indices de los sufijos en orden lexicografico:

$$S[SA[0]..] < S[SA[1]..] < \dots < S[SA[n-1]..].$$

El arreglo inverso rank cumple:

$$\text{rank}[i] = p \iff SA[p] = i.$$

El arreglo LCP guarda:

$$\text{LCP}[p] = \text{lcp}(S[SA[p-1]..], S[SA[p]..]).$$

7.2 Ejemplo: banana

Los sufijos de **banana** ordenados son:

$$\mathbf{a}(5), \mathbf{ana}(3), \mathbf{anana}(1), \mathbf{banana}(0), \mathbf{na}(4), \mathbf{nana}(2).$$

Entonces:

$$SA = [5, 3, 1, 0, 4, 2].$$

Como el sufijo que empieza en 1 es **anana** y ocupa la posicion 2, $\text{rank}[1] = 2$.

7.3 Manber-Myers

La idea es ordenar por prefijos de longitud $1, 2, 4, 8, \dots$. Si ya conocemos el rank de prefijos de longitud len , entonces el prefijo de longitud $2len$ del sufijo i se representa por:

$$(\text{rank}[i], \text{rank}[i + len]).$$

Ordenar estos pares ordena los prefijos de longitud $2len$. Pares iguales reciben el mismo nuevo rank.

7.4 Kasai

Kasai construye LCP en tiempo lineal despues de tener SA y rank. Para cada posicion i , mira el sufijo anterior en el orden lexicografico:

$$j = SA[\text{rank}[i] - 1].$$

Si el LCP anterior era h , al pasar de i a $i+1$ el LCP baja como mucho en 1, por lo que se empieza desde $\max(h-1, 0)$ y se avanza mientras coincidan caracteres.

7.5 Formula clave con RMQ

Si $a = \text{rank}[i] < b = \text{rank}[j]$, entonces:

$$\text{lcp}(i, j) = \min(\text{LCP}[a+1], \text{LCP}[a+2], \dots, \text{LCP}[b]).$$

Esto convierte LCP entre dos sufijos arbitrarios en una consulta RMQ.

7.6 Complejidad

Manber-Myers con sort comun cuesta $O(n \log^2 n)$; optimizado con counting/radix por ranks cuesta $O(n \log n)$. Kasai cuesta $O(n)$. El RMQ sobre LCP cuesta $O(n \log n)$ memoria y build, y responde en $O(1)$.

8 Aplicaciones obligatorias de Suffix Array

8.1 Buscar y contar ocurrencias de un patron

Todos los sufijos que empiezan con un patron P forman un intervalo contiguo en SA. Se hacen dos búsquedas binarias:

- L : primer sufijo lexicograficamente $\geq P$.
- R : primer sufijo que ya no tiene prefijo P .

Si $L = R$, el patron no aparece. Si no:

$$\#occ(P) = R - L, \quad posiciones = SA[L], \dots, SA[R - 1].$$

Complejidad simple por patron: $O(|P| \log n)$. Con comparaciones aceleradas por LCP puede mejorarse, pero para examen la idea central es el intervalo.

8.2 Subcadenas distintas

Cada subcadena es prefijo de algun sufijo. El sufijo $SA[i]$ aporta $n - SA[i]$ prefijos, pero los primeros $LCP[i]$ ya aparecieron en el sufijo anterior. Por tanto:

$$\#subcadenas\ distintas = \sum_i ((n - SA[i]) - LCP[i]) = \frac{n(n+1)}{2} - \sum_i LCP[i].$$

8.3 Longest Common Substring

Para dos cadenas A, B , construir:

$$T = A\#B\$,$$

donde $\#, \$$ no aparecen en las cadenas. En el suffix array de T , se revisan vecinos de origen distinto. El mayor LCP entre vecinos de distinto origen es una subcadena comun maxima.

Correctitud: cualquier subcadena comun es prefijo de un sufijo de A y de un sufijo de B . En el bloque de sufijos que comparten ese prefijo debe existir algun par vecino de origen distinto; por eso revisar vecinos basta.

8.4 Comparar y ordenar subcadenas

Para comparar $A = S[l_1..r_1]$ y $B = S[l_2..r_2]$, sea:

$$x = \min(\text{lcp}(l_1, l_2), |A|, |B|).$$

Si $x < \min(|A|, |B|)$, decide el primer caracter distinto:

$$S[l_1 + x] \quad \text{vs.} \quad S[l_2 + x].$$

Si $x = \min(|A|, |B|)$, una es prefijo de la otra y gana la mas corta. Si son identicas y el statement pide desempate por (l, r) , se compara el par original. Con SA+LCP+RMQ, cada comparacion cuesta $O(1)$; ordenar q subcadenas cuesta $O(q \log q)$.

8.5 Bordes de todas las subcadenas

Un borde es una cadena que es prefijo y sufijo. Si el problema pide bordes de **todas** las subcadenas, no alcanza KMP sobre la cadena completa. Cada borde no vacio de longitud b corresponde a dos ocurrencias iguales de longitud b en los extremos de alguna subcadena. Por tanto:

$$ans = \frac{n(n+1)}{2} + \sum_{i < j} \text{lcp}(i, j),$$

donde el primer termino cuenta el borde vacio si el statement lo incluye. La suma de LCP de todos los pares de sufijos equivale a la suma de minimos de todos los subarreglos del arreglo LCP, calculable con pila monotona en $O(n)$ despues de construir SA + LCP.

9 Árboles estáticos: LCA, HLD, Imperial Roads y Fools and Roads

9.1 LCA con binary lifting

$$up[v][j] = \text{ancestro de } v \text{ a distancia } 2^j.$$

Si se necesitan máximos en caminos:

$$mx[v][j] = \max \text{ peso de arista al subir de } v \text{ hacia } up[v][j].$$

Para responder $maxEdge(u, v)$, se igualan profundidades acumulando máximos; luego se suben ambos nodos desde potencias grandes hacia pequeñas hasta llegar al LCA.

$$T_{pre} = O(n \log n), \quad T_{query} = O(\log n), \quad M = O(n \log n).$$

9.2 HLD

HLD marca como pesado al hijo de mayor subarbol. Toda arista no pesada es liviana. Si se baja por una arista liviana, el tamaño del subarbol se reduce al menos a la mitad; por eso cualquier camino raíz-nodo cruza $O(\log n)$ aristas livianas. Entonces un camino $u \rightarrow v$ se descompone en $O(\log n)$ segmentos contiguos sobre un arreglo base.

Cuando usar HLD. Consultas o updates sobre caminos de un árbol fijo: suma, máximo, mínimo, pintar, agregar valor, comparar segmentos, etc. Con un segment tree, cada segmento cuesta $O(\log n)$, por lo que:

$$T_{path} = O(\log^2 n).$$

9.3 Imperial Roads

Problema tipo: grafo conectado ponderado. Cada consulta obliga a incluir una arista $e = (u, v, w)$ en el árbol de costo mínimo que conecta todos los vértices.

Algoritmo:

1. Construir MST T con Kruskal. Sea W su peso.
2. Preprocesar T para $maxEdge(u, v)$ con LCA o HLD.
3. Para query $e = (u, v, w)$:

$$ans = W + w - maxEdge_T(u, v).$$

Correctitud: al agregar e al MST aparece un único ciclo formado por e y el camino $u \rightarrow v$ en T . Para conservar un árbol que incluya e , hay que eliminar una arista de ese ciclo. El costo se minimiza eliminando la arista de mayor peso del camino original. Esta es una aplicación directa de la propiedad de ciclo del MST.

Complejidad:

$$T = O(m \log m + n \log n + q \log n), \quad M = O(n \log n + m).$$

9.4 Fools and Roads

Problema tipo: en un árbol, muchas consultas (u, v) . Para cada arista, contar cuántas rutas $u \rightarrow v$ la usan.

Algoritmo de diferencia en árbol:

$$cnt[u] += 1, \quad cnt[v] += 1, \quad cnt[lca(u, v)] -= 2.$$

Luego se hace DFS postorder:

$$sub[x] = cnt[x] + \sum_{\text{hijo } y} sub[y].$$

El valor $sub[x]$ es la cantidad de caminos que usan la arista $(parent[x], x)$.

Correctitud: cada query agrega dos unidades de flujo en los extremos. Ese flujo sube por el árbol y se cancela en el LCA. Por tanto, una arista recibe flujo exactamente cuando está en el camino entre los extremos.

Complejidad:

$$T = O(n \log n + q \log n + n), \quad M = O(n \log n).$$

10 Dynamic Graphs: rollback DSU y estructuras dinamicas

10.1 Offline dynamic connectivity

Si hay operaciones de agregar y borrar aristas, pero conocemos toda la secuencia antes de responder, podemos resolver offline.

Transformacion clave. Cada arista tiene intervalos de vida $[l, r]$ en los que esta activa. Insertamos ese intervalo en los nodos de un segment tree sobre el tiempo que cubren exactamente $[l, r]$.

```
dfs(node):
    snap = DSU.snapshot()
    for edge in node.edges:
        DSU.union(edge.u, edge.v)
    if node is leaf:
        answer queries at that time
    else:
        dfs(left); dfs(right)
    DSU.rollback(snap)
```

10.2 Rollback DSU

Un DSU con rollback guarda en una pila los cambios hechos por cada union. Normalmente no usa path compression, porque comprimir caminos modifica muchos padres y seria dificil deshacer. Usa union by size/rank.

Si ademas se pide suma de componente, cada raiz guarda:

$$sum[root] = \sum_{v \in component(root)} value[v].$$

Al unir componentes, se suma. Para rollback, se guardan los valores antiguos de *parent*, *size* y *sum*.

10.3 Correctitud

Un intervalo de vida $[l, r]$ se descompone en $O(\log q)$ nodos del segment tree. Para una hoja t , el camino desde la raiz hasta esa hoja contiene exactamente los nodos cuyos intervalos cubren t . Por eso, al llegar a la hoja, el DSU contiene exactamente las aristas activas en el tiempo t . El rollback restaura el estado antes de explorar otra rama.

10.4 Complejidad

Si hay m intervalos de vida y q tiempos:

$$T = O(m \log q \cdot T_{\text{union}}), \quad M = O(m \log q + n).$$

Con union by size sin path compression, T_{union} es $O(\log n)$ en una cota simple, o casi constante en practica. Lo importante para examen es justificar el factor $\log q$ por el segment tree de tiempo.

10.5 Euler-tour tree vs link-cut tree

Caso	Estructura	Idea
Bosque dinamico, conectividad, size, suma de componente	Euler-tour tree	tour Euler en BST balanceado; link-cut son split/merge
Bosque dinamico, max/sum/min en camino	Link-cut tree	preferred paths en splay; access expone un camino
Grafo general offline	Rollback DSU	intervalos de vida sobre tiempo
Solo inserciones	DSU normal	componentes monotonicamente se unen
Solo borrados	Procesar al revés	los borrados se vuelven inserciones

Link-cut tree en una frase. $\text{access}(v)$ reorganiza los preferred paths para que el camino desde la raíz representada hasta v quede en un splay, donde el agregado del camino esta en la raíz.

Euler-tour tree en una frase. Representa cada arbol por la secuencia de su tour Euler guardada en un BST balanceado. Si dos vertices pertenecen al mismo BST, estan conectados.

11 Integer structures

11.1 Modelo y problema predecesor

Se mantiene un conjunto S de n enteros de w bits. El universo es:

$$U = \{0, 1, \dots, 2^w - 1\}, \quad u = 2^w.$$

Operaciones:

$$insert(x), delete(x), member(x), pred(x), succ(x).$$

En modelo de comparaciones, predecesor cuesta $O(\log n)$ con un BST. En word RAM se usan bits y el universo para mejorar.

11.2 van Emde Boas

vEB divide el universo en coordenadas high/low:

$$x = (high(x), low(x)),$$

donde cada parte tiene tamaño $\sqrt{u}$. Guarda:

- min, max ;
- clusters para las partes altas;
- un summary que indica que clusters no estan vacios.

La recurrencia es:

$$T(u) = T(\sqrt{u}) + O(1).$$

Como $u = 2^w$, cada llamada divide el numero de bits por 2:

$$T(w) = T(w/2) + O(1) = O(\log w) = O(\log \log u).$$

El problema es el espacio:

$$M = \Theta(u)$$

en la version basica.

11.3 Y-fast trees

Y-fast reduce el espacio a lineal. La idea es:

- mantener representantes de buckets en una estructura tipo X-fast;
- cada bucket tiene tamaño $O(w)$ y se guarda con una estructura balanceada;
- split/merge mantiene buckets de tamaño controlado.

Resultado teorico:

$$T_{op} = O(\log w) = O(\log \log u) \text{ esperado/amortizado}, \quad M = O(n).$$

Si el universo es enorme y n es pequeno, Y-fast es mejor respuesta que vEB basico por espacio.

11.4 Fusion trees

Fusion trees usan branching alto $B = w^{1/5}$. Dentro de un nodo se guardan llaves y se extraen bits criticos del trie conceptual para formar sketches. Esos sketches se empaquetan en palabras y se comparan en paralelo. La altura baja a:

$$O(\log_B n) = O(\log_w n).$$

En examen, mencionar que depende de operaciones word RAM fuertes; algunas versiones usan multiplicacion, lo cual importa si el modelo es AC0 RAM.

11.5 Ordenamiento de enteros

La cota $\Omega(n \log n)$ aplica al modelo de comparaciones. Con enteros se pueden mirar digitos.

Counting sort. Si las llaves estan en universo U_d :

$$T = O(n + U_d), \quad M = O(U_d).$$

Radix sort. Partir cada llave en digitos de b bits. Cada pasada usa counting sort estable:

$$T = O\left((n + 2^b) \left\lceil \frac{w}{b} \right\rceil\right).$$

Si $b = \Theta(\log n)$, entonces $2^b = \Theta(n)$ y:

$$T = O\left(n \frac{w}{\log n}\right).$$

Si $w = O(\log n)$, se obtiene tiempo lineal.

Signature sort. Para enteros grandes, se reducen llaves a firmas/ hashes de $O(\log n)$ bits por niveles, y luego se aplica packed/radix sort. La idea teorica es reducir los bits relevantes sin perder el orden.

12 Patrones de respuesta para problemas Racso

12.1 Si aparece un problema de strings

Primero intenta ubicarlo en uno de estos moldes:

- Buscar o contar patrones: intervalo en suffix array.
- Numero de subcadenas distintas: $\frac{n(n+1)}{2} - \sum \text{LCP}$.
- Longest common substring: SA de $A\#B$ y max LCP entre origen distinto.
- Ordenar/comparar subcadenas: $\text{lcp}(i, j)$ con RMQ sobre LCP.
- Muchos patrones en un texto: Aho-Corasick para ocurrencias, SA si el texto fijo es lo central.
- Periodicidad: igualdad de bloques via LCP; si es avanzado, mencionar runs.

12.2 Si aparece un problema de arboles

- Una ruta especifica $u \rightarrow v$: LCA o HLD.
- Muchas rutas y contar aristas usadas: diferencia en arbol.
- Maximo/minimo en path de un arbol fijo: HLD o binary lifting augmentado.
- Contar pares de nodos por distancia: centroid decomposition.
- Subarboles: Euler tour timestamps.

12.3 Si aparece dynamic graph

- Solo inserciones: DSU.
- Solo borrados: procesar al reves.
- Inserciones y borrados offline: segment tree sobre tiempo + rollback DSU.
- Bosque dinamico online con path aggregates: link-cut tree.
- Bosque dinamico online con componente/subtree aggregates: Euler-tour tree.
- Grafo general online: spanning forests por niveles + Euler-tour trees; es mas teorico.

12.4 Si aparece integer

Siempre empieza definiendo $n, w, u = 2^w$ y el modelo. Luego:

- Predecessor con universo moderado: vEB, $O(\log \log u)$, espacio $\Theta(u)$.
- Predecessor con espacio lineal: Y-fast, $O(\log w)$, espacio $O(n)$.
- Predecessor teorico rapido: Fusion tree, $O(\log_w n)$.
- Sorting de enteros: counting/radix si w es pequeno; signature si w es grande.

13 Errores que quitan puntos

- Confundir rango cerrado $[l, r]$ con medio abierto $[l, r)$ en RMQ.
- Decir “uso suffix array” sin explicar intervalo contiguo o LCP.
- En HLD con valores en aristas, olvidar excluir la posicion del LCA.
- Usar path compression en rollback DSU.
- Para Imperial Roads, olvidar que se fuerza una arista y se quita la maxima del ciclo.
- Para bordes, resolver solo bordes de la cadena completa cuando el problema pide todas las subcadenas.

- No mencionar memoria. En integer, vEB tiene espacio $\Theta(u)$, no lineal.
- Usar int para cantidades que pueden ser $\Theta(n^2)$: subcadenas, pares de nodos, sumas de LCP.

14 Resumen final de formulas y complejidades

Esta tabla es para repasar en los ultimos minutos antes de entrar.

Tema	Formula / idea que debes recordar	Costo
Sparse table	$st[k][i] = op(st[k-1][i], st[k-1][i+2^{k-1}])$. Query con dos bloques de tamaño 2^k .	Build $O(n \log n)$, query $O(1)$, memoria $O(n \log n)$.
LCA binary lifting	$up[v][j]$ ancestro 2^j . Para max edge, guardar $mx[v][j]$.	Pre $O(n \log n)$, query $O(\log n)$.
HLD	Light edge reduce subarbol al menos a la mitad. Path $u \rightarrow v$ son $O(\log n)$ segmentos.	Pre $O(n)$, path con segtree $O(\log^2 n)$.
Suffix array	$rank[i]$ es posicion del sufijo i en SA. $LCP[p] = lcp(SA[p-1], SA[p])$.	SA $O(n \log n)$ o $O(n \log^2 n)$, Kasai $O(n)$.
LCP arbitrario	Si $a = rank[i] < b = rank[j]$, $lcp(i, j) = \min LCP[a+1..b]$.	RMQ build $O(n \log n)$, query $O(1)$.
Subcadenas distintas	$\# = \frac{n(n+1)}{2} - \sum_i LCP[i]$.	$O(n)$ despues de SA+LCP.
Imperial Roads	$ans = W + w - \max Edge_T(u, v)$.	Kruskal $O(m \log m)$, query $O(\log n)$.
Fools and Roads	$cnt[u]++, cnt[v]++, cnt[lca(u, v)]-- = 2$; postorder acumula.	$O((n+q) \log n)$.
Centroid paths $\leq C$	En centroid: contar pares de distancias con two pointers; restar pares del mismo subarbol.	$O(n \log^2 n)$, memoria $O(n)$.
Rollback DSU	Arista activa en intervalo $[l, r]$; guardar en segment tree de tiempo; DFS con snapshot/rollback.	$O(m \log q)$ uniones, memoria $O(m \log q + n)$.
vEB	$T(u) = T(\sqrt{u}) + O(1) = O(\log \log u)$.	Tiempo $O(\log \log u)$, memoria $\Theta(u)$.
Radix sort	$O((n+2^b) \lceil w/b \rceil)$, elegir $b = \Theta(\log n)$.	$O(nw / \log n)$, lineal si $w = O(\log n)$.

Frase final para casi toda respuesta. “La estructura es correcta porque mantiene exactamente la informacion que la consulta necesita: minimo de rangos canonicos, prefijo comun de sufijos, maximo en un camino, componentes activas en un tiempo, o bits relevantes del universo. El algoritmo no enumera objetos cuadraticos; los comprime en intervalos, caminos, clusters, buckets o descomposiciones balanceadas.”